

Q.N. (1) Restaurant Information System - DFD

DFD Level-0 (Context Diagram):

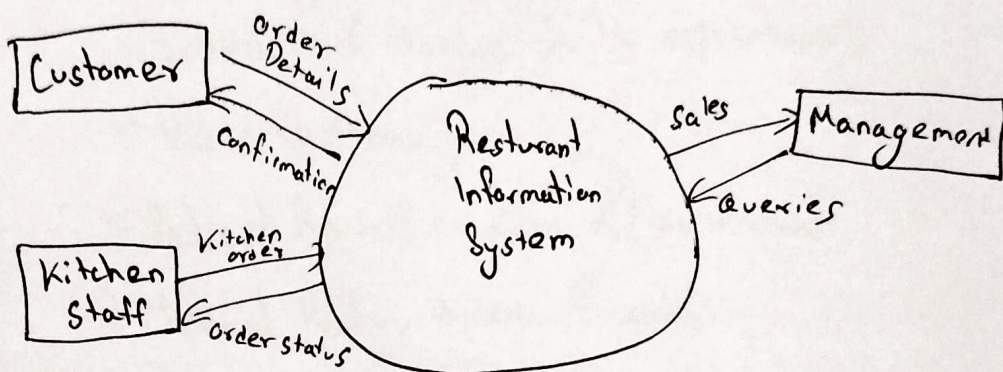
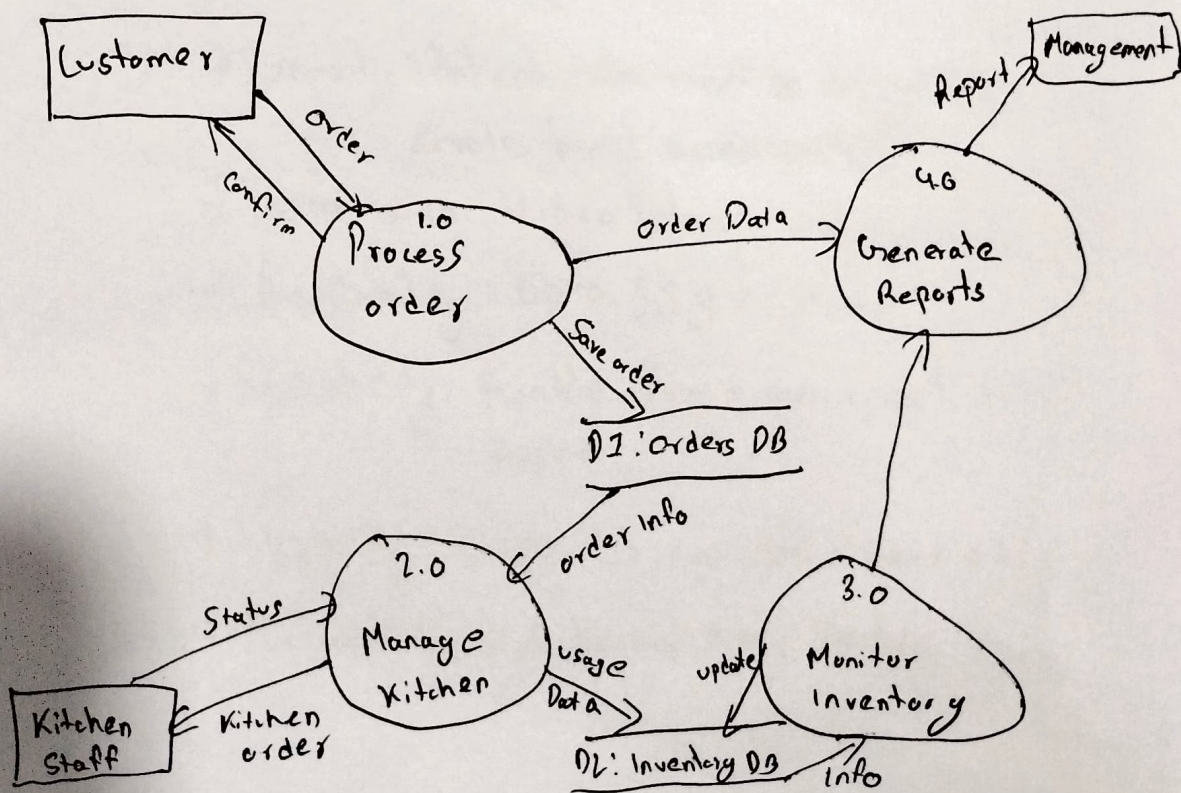


Fig: DFD Level-0 for Restaurant IS

DFD Level-1 for Restaurant Information System:



Q.N. (2) Telemedicine Project

⇒ (a) Requirements

⊛ Functional Requirements

- User Registration & Login (Doctor, Patients, Admins)
- Schedule & Manage Doctor appointments
- Video Conferencing
- Patient Record Creation & Maintenance
- Medical History Access & update
- Prescription Generation by Doctor
- Follow-up Appointment Management
- Central Server Data Storage & Retrieval
- Report Generation for MoH

⊛ Non-Functional Requirements

- Security: Patient data must be encrypted
Role-based access control
- Performance: Video latency < 150ms
- Availability: upto 99.9% uptime
- Scalability: Handle 500+ concurrent video session
- Usability: Simple UI, accessible in rural areas
- Reliability: Autobackup every 24 hrs.

(b) DFD Diagram for Telemedicine System

DFD Level-0 (Context Diagram)

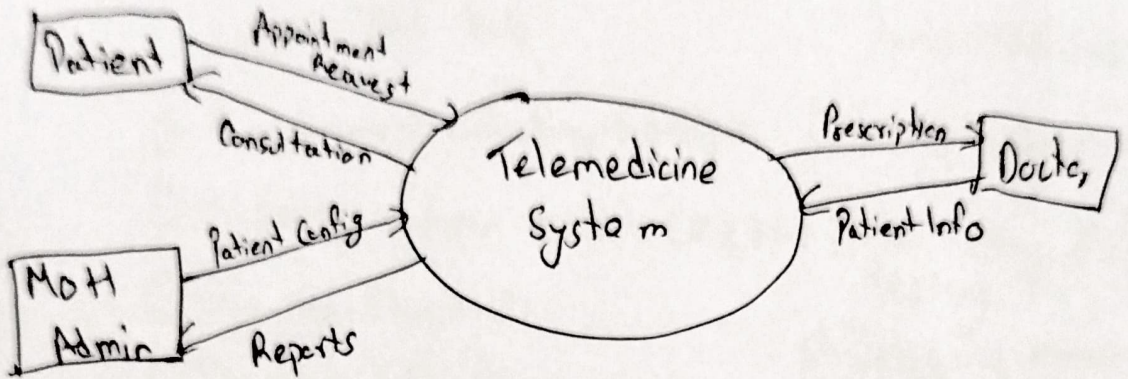
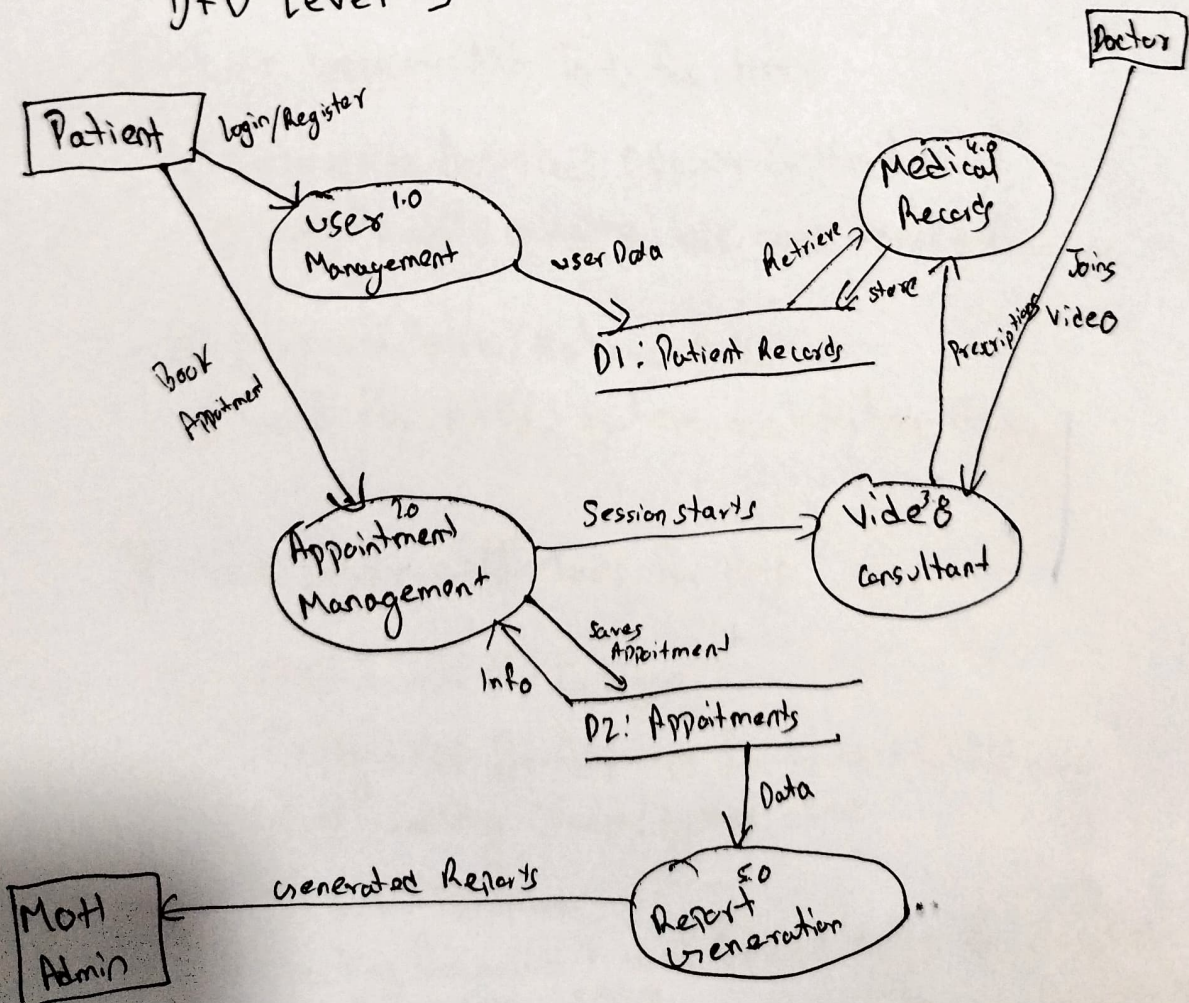


Fig: DFD Level-0

DFD Level-1



Q.N. (2) Hospital Management System - Case Tools

⇒ 1) Upper-CASE vs Lower-CASE Activities,

Upper-CASE Tools

- ① Requirement collections & analysis
- ② Preparing system models (DFD, ER)
- ③ Drawing flowcharts
- ④ Creating UML Diagrams
- ⑤ Database schema Design

Lower-CASE Tools

- ① Writing/Loading source code
- ② Unit testing & integration testing
- ③ Debugging errors
- ④ Version control & CI/CD
- ⑤ Deploying the system to hospital server.

2) Two Upper-CASE Tools for HMS

a) Enterprise Architect (Sparx Systems)

→ Used for UML modeling, use case diagram & so on.

b) Microsoft Visio / Rational Rose

→ Used for DFDs, system architecture diagram.

3) Two Lower-~~case~~^{CASE} Tools for HMS

a) IntelliJ IDEA / Eclipse

→ Integrated Development Environment (IDE) used for writing Java/Python code.

b) JUnit / Selenium

→ Testing framework used for unit testing & automated

4) Importance of Upper-CASE Tools Before Coding:

- ① Clear Blueprint: Diagram give developer a clear roadmap before writing code.
- ① Reduces Rework: Design flaws caught early cost less to fix than post-coding errors.
- ① Stakeholder Communication: Visual models help non-technical users to validate requirements.
- ① Better Architecture: Proper design ensures the system is scalable and maintainable.
- ① Documentation: UML/DFD diagrams serves as official documentation for system lifecycle.

5) Issues if Coding starts without proper Design:

- ④ Incomplete Requirements: Developer may miss key features like billing integration or doctor scheduling.
- ④ Poor System Architecture: Without design, the codebase becomes tightly coupled and hard to maintain.
- ④ High cost of changes: Fixing design flaws discovered during/after coding is 10x more expensive.
- ④ Security Vulnerabilities: Lack of security planning leads to data breaches.
- ④ Project Failure: May completely fail to meet user needs.